HTML 5.2 1
W3C Recommendation, 14 December 2017 1
1. Introduction § 1
1.1. Background § 1
1.2. Audience § 1
1.3. Scope § 2
1.4. History § 2
1.5. Design notes § 4
1.5.1. Serializability of script execution § 4
1.5.2. Compliance with other specifications § 4
1.5.3. Extensibility § 4
1.6. HTML vs XML Syntax § 5
1.7. Structure of this specification § 5
1.7.1. How to read this specification § 6
1.7.2. Typographic conventions § 7
1.8. Privacy concerns § 8
1.9. A quick introduction to HTML § 9
1.9.1. Writing secure applications with HTML § 11
1.9.2. Common pitfalls to avoid when using the scripting APIs § 13
1.9.3. How to catch mistakes when writing HTML: validators and conformance checkers § 14
1.10. Conformance requirements for authors § 14
1.10.1. Presentational markup § 15
1.10.2. Syntax errors § 15
1.10.3. Restrictions on content models and on attribute values § 19
1.11. Suggested reading § 22



HTML 5.2

W3C Recommendation, 14 December 2017

§ 1. Introduction

§ 1.1. Background

This section is non-normative.

HTML is the World Wide Web's core markup language. Originally, HTML was primarily designed as a language for semantically describing scientific documents. Its general design, however, has enabled it to be adapted, over the subsequent years, to describe a number of other types of documents and even applications.

§ 1.2. Audience

This section is non-normative.

This specification is intended for authors of documents and scripts that use the features defined in this

specification, implementors of tools that operate on pages that use the features defined in this specification, and individuals wishing to establish the correctness of documents or implementations with respect to the requirements of this specification.

This document is probably not suited to readers who do not already have at least a passing familiarity with Web technologies, as in places it sacrifices clarity for precision, and brevity for completeness. More approachable tutorials and authoring guides can provide a gentler introduction to the topic.

In particular, familiarity with the basics of DOM is necessary for a complete understanding of some of the more technical parts of this specification. An understanding of Web IDL, HTTP, XML, Unicode, character encodings, JavaScript, and CSS will also be helpful in places but is not essential.

§ 1.3. Scope

This section is non-normative.

This specification is limited to providing a semantic-level markup language and associated semantic-level scripting APIs for authoring accessible pages on the Web ranging from static documents to dynamic applications.

The scope of this specification does not include providing mechanisms for media-specific customization of presentation (although default rendering rules for Web browsers are included at the end of this specification, and several mechanisms for hooking into CSS are provided as part of the language).

The scope of this specification is not to describe an entire operating system. In particular, hardware configuration software, image manipulation tools, and applications that users would be expected to use with high-end workstations on a daily basis are out of scope. In terms of applications, this specification is targeted specifically at applications that would be expected to be used by users on an occasional basis, or regularly but from disparate locations, with low CPU requirements. Examples of such applications include online purchasing systems, searching systems, games (especially multiplayer online games), public telephone books or address books, communications software (e-mail clients, instant messaging clients, discussion software), document editing software, etc.

§ 1.4. History

This section is non-normative.

For its first five years (1990-1995), HTML went through a number of revisions and experienced a number of extensions, primarily hosted first at CERN, and then at the IETF.

With the creation of the W3C, HTML's development changed venue again. A first abortive attempt at extending HTML in 1995 known as HTML 3.0 then made way to a more pragmatic approach known as HTML 3.2, which was completed in 1997. HTML 4.01 quickly followed later that same year.

The following year, the W3C membership decided to stop evolving HTML and instead begin work on an XML-based equivalent, called XHTML. This effort started with a reformulation of HTML 4.01 in XML, known as XHTML 1.0, which added no new features except the new serialization, and which was completed in 2000. After XHTML 1.0, the W3C's focus turned to making it easier for other working groups to extend XHTML, under the banner of XHTML Modularization. In parallel with this, the W3C also worked on a new language that was not compatible with the earlier HTML and XHTML languages, calling it XHTML 2.0.

Around the time that HTML's evolution was stopped in 1998, parts of the API for HTML developed by browser vendors were specified and published under the name DOM Level 1 (in 1998) and DOM Level 2 Core and DOM Level 2 HTML (starting in 2000 and culminating in 2003). These efforts then petered out,

with some DOM Level 3 specifications published in 2004 but the working group being closed before all the Level 3 drafts were completed.

In 2003, the publication of XForms, a technology which was positioned as the next generation of Web forms, sparked a renewed interest in evolving HTML itself, rather than finding replacements for it. This interest was borne from the realization that XML's deployment as a Web technology was limited to entirely new technologies (like RSS and later Atom), rather than as a replacement for existing deployed technologies (like HTML).

A proof of concept to show that it was possible to extend HTML 4.01's forms to provide many of the features that XForms 1.0 introduced, without requiring browsers to implement rendering engines that were incompatible with existing HTML Web pages, was the first result of this renewed interest. At this early stage, while the draft was already publicly available, and input was already being solicited from all sources, the specification was only under Opera Software's copyright.

The idea that HTML's evolution should be reopened was tested at a W3C workshop in 2004, where some of the principles that underlie the HTML work (described below), as well as the aforementioned early draft proposal covering just forms-related features, were presented to the W3C jointly by Mozilla and Opera. The proposal was rejected on the grounds that the proposal conflicted with the previously chosen direction for the Web's evolution; the W3C staff and membership voted to continue developing XML-based replacements instead.

Shortly thereafter, Apple, Mozilla, and Opera jointly announced their intent to continue working on the effort under the umbrella of a new venue called the WHATWG. A public mailing list was created, and the draft was moved to the WHATWG site. The copyright was subsequently amended to be jointly owned by all three vendors, and to allow reuse of the specification.

The WHATWG was based on several core principles, in particular that technologies need to be backwards compatible, that specifications and implementations need to match even if this means changing the specification rather than the implementations, and that specifications need to be detailed enough that implementations can achieve complete interoperability without reverse-engineering each other.

The latter requirement in particular required that the scope of the HTML specification include what had previously been specified in three separate documents: HTML 4.01, XHTML 1.1, and DOM Level 2 HTML. It also meant including significantly more detail than had previously been considered the norm.

In 2006, the W3C indicated an interest to participate in the development of HTML 5.0 after all, and in 2007 formed a working group chartered to work with the WHATWG on the development of the HTML specification. Apple, Mozilla, and Opera allowed the W3C to publish the specification under the W3C copyright, while keeping a version with the less restrictive license on the WHATWG site.

For a number of years, both groups then worked together under the same editor: Ian Hickson. In 2011, the groups came to the conclusion that they had different goals: the W3C wanted to draw a line in the sand for features for a HTML 5.0 Recommendation, while the WHATWG wanted to continue working on a Living Standard for HTML, continuously maintaining the specification and adding new features. In mid 2012, a new editing team was introduced at the W3C to take care of creating a HTML 5.0 Recommendation and prepare a Working Draft for the next HTML version.

Since then, the W3C Web Platform WG has been cherry picking patches from the WHATWG that resolved bugs registered on the W3C HTML specification or more accurately represented implemented reality in user agents. At time of publication of this document, patches from the WHATWG HTML specification have been merged until January 12, 2016. The W3C HTML editors have also added patches that resulted from discussions and decisions made by the W3C Web Platform WG as well a bug fixes from bugs not shared by the WHATWG.

A separate document is published to document the differences between the HTML specified in this

document and the language described in the HTML 4.01 specification. [HTML5-DIFF]

1.5. Design notes

This section is non-normative.

It must be admitted that many aspects of HTML appear at first glance to be nonsensical and inconsistent.

HTML, its supporting DOM APIs, as well as many of its supporting technologies, have been developed over a period of several decades by a wide array of people with different priorities who, in many cases, did not know of each other's existence.

Features have thus arisen from many sources, and have not always been designed in especially consistent ways. Furthermore, because of the unique characteristics of the Web, implementation bugs have often become de-facto, and now de-jure, standards, as content is often unintentionally written in ways that rely on them before they can be fixed.

Despite all this, efforts have been made to adhere to certain design goals. These are described in the next few subsections.

1.5.1. Serializability of script execution

This section is non-normative.

To avoid exposing Web authors to the complexities of multithreading, the HTML and DOM APIs are designed such that no script can ever detect the simultaneous execution of other scripts. Even with `workers`, the intent is that the behavior of implementations can be thought of as completely serializing the execution of all scripts in all browsing contexts.

1.5.2. Compliance with other specifications

This section is non-normative.

This specification interacts with and relies on a wide variety of other specifications. In certain circumstances, unfortunately, conflicting needs have led to this specification violating the requirements of these other specifications. Whenever this has occurred, the transgressions have each been noted as a "***willful violation***", and the reason for the violation has been noted.

1.5.3. Extensibility

This section is non-normative.

HTML has a wide array of extensibility mechanisms that can be used for adding semantics in a safe manner:

- Authors can use the `class` attribute to extend elements, effectively creating their own elements, while using the most applicable existing "real" HTML element, so that browsers and other tools that don't know of the extension can still support it somewhat well. This is the tack used by microformats, for example.
- Authors can include data for inline client-side scripts or server-side site-wide scripts to process using the `data-*=""` attributes. These are guaranteed to never be touched by browsers, and allow scripts to include data on HTML elements that scripts can then look for and process.

- Authors can use the `<meta name="" content="">` mechanism to include page-wide metadata by registering extensions to the predefined set of metadata names.
- Authors can use the `rel=""` mechanism to annotate links with specific meanings by registering extensions to the predefined set of link types. This is also used by microformats.
- Authors can embed raw data using the `<script type="">` mechanism with a custom type, for further handling by inline or server-side scripts.
- Authors can extend APIs using the JavaScript prototyping mechanism. This is widely used by script libraries, for instance.

§ 1.6. HTML vs XML Syntax

This section is non-normative.

This specification defines an abstract language for describing documents and applications, and some APIs for interacting with in-memory representations of resources that use this language.

The in-memory representation is known as "DOM HTML", or "the DOM" for short.

There are various concrete syntaxes that can be used to transmit resources that use this abstract language, two of which are defined in this specification.

The first such concrete syntax is the HTML syntax. This is the format suggested for most authors. It is compatible with most legacy Web browsers. If a document is transmitted with the `text/html` MIME type, then it will be processed as an HTML document by Web browsers. This specification defines the latest version of the HTML syntax, known simply as "HTML".

The second concrete syntax is the XHTML syntax, which is an application of XML. When a document is transmitted with an XML MIME type, such as `application/xhtml+xml`, then it is treated as an XML document by Web browsers, to be parsed by an XML processor. Authors are reminded that the processing for XML and HTML differs; in particular, even minor syntax errors will prevent a document labeled as XML from being rendered fully, whereas they would be ignored in the HTML syntax. This specification defines the latest version of the XHTML syntax, known simply as "XHTML".

The DOM, the HTML syntax, and the XHTML syntax cannot all represent the same content. For example, namespaces cannot be represented using the HTML syntax, but they are supported in the DOM and in the XHTML syntax. Similarly, documents that use the `<noscript>` feature can be represented using the HTML syntax, but cannot be represented with the DOM or in the XHTML syntax. Comments that contain the string "`-->`" can only be represented in the DOM, not in the HTML and XHTML syntaxes.

§ 1.7. Structure of this specification

This section is non-normative.

This specification is divided into the following major sections:

§1 Introduction
Non-normative materials providing a context for the HTML specification.

§2 Common infrastructure
The conformance classes, algorithms, definitions, and the common underpinnings of the rest of the specification.

§3 Semantics, structure, and APIs of HTML documents

Documents are built from elements. These elements form a tree using the DOM. This section defines the features of this DOM, as well as introducing the features common to all elements, and the concepts used in defining elements.

§4 The elements of HTML
Each element has a predefined meaning, which is explained in this section. Rules for authors on how to use the element, along with user agent requirements for how to handle each element, are also given. This includes large signature features of HTML such as video playback and subtitles, form controls and form submission, and a 2D graphics API known as the HTML canvas.

§5 User interaction
HTML documents can provide a number of mechanisms for users to interact with and modify content, which are described in this section, such as how focus works, and drag-and-drop.

§6 Loading Web pages
HTML documents do not exist in a vacuum — this section defines many of the features that affect environments that deal with multiple pages, such as Web browsers and offline caching of Web applications.

§7 Web application APIs
This section introduces basic features for scripting of applications in HTML.

§8 The HTML syntax
§9 The XML syntax
All of these features would be for naught if they couldn't be represented in a serialized form and sent to other people, and so these sections define the syntaxes of HTML and XHTML, along with rules for how to parse content using those syntaxes.

§10 Rendering
This section defines the default rendering rules for Web browsers.

There are also some appendices, listing §11 Obsolete features and §12 IANA considerations, and several indices.

§ 1.7.1. How to read this specification

This specification should be read like all other specifications. First, it should be read cover-to-cover, multiple times. Then, it should be read backwards at least once. Then it should be read by picking random sections from the contents list and following all the cross-references.

As described in the conformance requirements section below, this specification describes conformance criteria for a variety of conformance classes. In particular, there are conformance requirements that apply to *producers*, for example authors and the documents they create, and there are conformance requirements that apply to *consumers*, for example Web browsers. They can be distinguished by what they are requiring: a requirement on a producer states what is allowed, while a requirement on a consumer states how software is to act.

¶ EXAMPLE COUNTER(EXAMPLE) For example, "the `foo` attribute's value must be a valid integer" is a requirement on producers, as it lays out the allowed values; in contrast, the requirement "the `foo` attribute's value must be parsed using the rules for parsing integers" is a requirement on consumers, as it describes how to process the content.

Requirements on producers have no bearing whatsoever on consumers.

¶ EXAMPLE COUNTER(EXAMPLE) Continuing the above example, a requirement stating that a particular attribute's value is constrained to being a valid integer emphatically does *not* imply anything about the requirements on consumers. It might be that the consumers are in fact required to treat the attribute as an opaque string, completely unaffected by whether the value conforms to the requirements or not. It might be (as in the previous example) that the consumers are required to parse the value using specific rules that define how invalid (non-numeric in this case) values are to be processed.

§ 1.7.2. Typographic conventions

This is a definition, requirement, or explanation.

This is a note.

¶ EXAMPLE COUNTER(EXAMPLE)This is an example.

This is an open issue.

This is a warning.

```
interface Example {
    // this is an IDL definition
};
```

variable = ***object***`.method([`***optionalArgument***`])`
This is a note to authors describing the usage of an interface.

```
'/* this is a CSS fragment */'
```

The defining instance of a term is marked up like ***this***. Uses of that term are marked up like this or like this.

The defining instance of an element, attribute, or API is marked up like ***`this`***. References to that element, attribute, or API are marked up like `<this>`.

Other code fragments are marked up `like this`.

Byte sequences with bytes in the range 0x00 to 0x7F, inclusive, are marked up like `this`.

Variables are marked up like *this*.

In an algorithm, steps in synchronous sections are marked with .

In some cases, requirements are given in the form of lists with conditions and corresponding requirements. In such cases, the requirements that apply to a condition are always the first set of requirements that follow the condition, even in the case of there being multiple sets of conditions for those requirements. Such cases are presented as follows:

This is a condition
This is another condition
This is the requirement that applies to the conditions above.

This is a third condition
This is the requirement that applies to the third condition.

§ 1.8. Privacy concerns

This section is non-normative.

Some features of HTML trade user convenience for a measure of user privacy.

In general, due to the Internet's architecture, a user can be distinguished from another by the user's IP address. IP addresses do not perfectly match to a user; as a user moves from device to device, or from network to network, their IP address will change; similarly, NAT routing, proxy servers, and shared computers enable packets that appear to all come from a single IP address to actually map to multiple users. Technologies such as onion routing can be used to further anonymize requests so that requests from a single user at one node on the Internet appear to come from many disparate parts of the network.

However, the IP address used for a user's requests is not the only mechanism by which a user's requests could be related to each other. Cookies, for example, are designed specifically to enable this, and are the basis of most of the Web's session features that enable you to log into a site with which you have an account.

There are other mechanisms that are more subtle. Certain characteristics of a user's system can be used to distinguish groups of users from each other; by collecting enough such information, an individual user's browser's "digital fingerprint" can be computed, which can be as good, if not better, as an IP address in ascertaining which requests are from the same user.

Grouping requests in this manner, especially across multiple sites, can be used for both benign (and even arguably positive) purposes, as well as for malevolent purposes. An example of a reasonably benign purpose would be determining whether a particular person seems to prefer sites with dog illustrations as opposed to sites with cat illustrations (based on how often they visit the sites in question) and then automatically using the preferred illustrations on subsequent visits to participating sites. Malevolent purposes, however, could include governments combining information such as the person's home address (determined from the addresses they use when getting driving directions on one site) with their apparent political affiliations (determined by examining the forum sites that they participate in) to determine whether the person should be prevented from voting in an election.

Since the malevolent purposes can be remarkably evil, user agent implementors are encouraged to consider how to provide their users with tools to minimize leaking information that could be used to fingerprint a user.

Unfortunately, as the first paragraph in this section implies, sometimes there is great benefit to be derived from exposing the very information that can also be used for fingerprinting purposes, so it's not as easy as simply blocking all possible leaks. For instance, the ability to log into a site to post under a specific identity requires that the user's requests be identifiable as all being from the same user. More subtly, though, information such as how wide text is, which is necessary for many effects that involve drawing text onto a canvas (e.g., any effect that involves drawing a border around the text) also leaks information that can be used to group a user's requests. (In this case, by potentially exposing, via a brute force search, which fonts a user has installed, information which can vary considerably from user to user.)

Features in this specification which can be ***used to fingerprint the user*** are marked as this paragraph is.

Other features in the platform can be used for the same purpose, though, including, though not limited to:

- The exact list of which features a user agents supports.
- The maximum allowed stack depth for recursion in script.
- Features that describe the user's environment, like Media Queries and the `Screen` object. [MEDIAQ]

[CSSOM-VIEW]

- The user's time zone.

§ 1.9. A quick introduction to HTML

This section is non-normative.

A basic HTML document looks like this:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Sample page</title>
  </head>
  <body>
    <h1>Sample page</h1>
    <p>This is a <a href="demo.html">simple</a> sample.</p>
    <!-- this is a comment -->
  </body>
</html>
```

HTML documents consist of a tree of elements and text. Each element is denoted in the source by a start tag, such as "`<body>`", and an end tag, such as "`</body>`". (Certain start tags and end tags can in certain cases be omitted and are implied by other tags.)

Tags have to be nested such that elements are all completely within each other, without overlapping:

```
<p>This is <em>very <strong>wrong</em>!</strong></p>
```

```
<p>This <em>is <strong>correct</strong>.</em></p>
```

This specification defines a set of elements that can be used in HTML, along with rules about the ways in which the elements can be nested.

Elements can have attributes, which control how the elements work. In the example below, there is a hyperlink, formed using the `<a>` element and its `href` attribute:

```
<a href="demo.html">simple</a>
```

Attributes are placed inside the start tag, and consist of a name and a value, separated by an "=" character. The attribute value can remain unquoted if it doesn't contain space characters or any of " ' ` = < or >. Otherwise, it has to be quoted using either single or double quotes. The value, along with the "=" character, can be omitted altogether if the value is the empty string.

```
<!-- empty attributes -->
<input name=address disabled>
<input name=address disabled="">

<!-- attributes with a value -->
<input name=address maxlength=200>
<input name=address maxlength='200'>
<input name=address maxlength="200">
```

HTML user agents (e.g., Web browsers) then parse this markup, turning it into a DOM (Document Object Model) tree. A DOM tree is an in-memory representation of a document.

DOM trees contain several kinds of nodes, in particular a `DocumentType` node, `Element` nodes, `Text` nodes, `Comment` nodes, and in some cases `ProcessingInstruction` nodes.

The markup snippet at the top of this section would be turned into the following DOM tree:

- DOCTYPE: `html`
- `<html>`
  - `<head>`
    - `#text`:
    - `<title>`
      - `#text`: Sample page
    - `#text`:
  - `#text`:
  - `<body>`
    - `#text`:
    - `<h1>`
      - `#text`: Sample page
    - `#text`:
    - `<p>`
      - `#text`: This is a
      - `<a>` `href`="demo.html"
        - `#text`: simple
      - `#text`: sample.
    - `#text`:
    - `#comment`: this is a comment
    - `#text`:

The `document` element of this tree is the `<html>` element, which is the element always found in that position in HTML documents. It contains two elements, `<head>` and `<body>`, as well as a `Text` node between them.

There are many more `Text` nodes in the DOM tree than one would initially expect, because the source contains a number of spaces (represented here by "␣") and line breaks ("⏎") that all end up as `Text` nodes in the DOM. However, for historical reasons not all of the spaces and line breaks in the original markup appear in the DOM. In particular, all the white space before `<head>` start tag ends up being dropped silently, and all the white space after the `<body>` end tag ends up placed at the end of the `<body>`.

The `<head>` element contains a `<title>` element, which itself contains a `Text` node with the text "Sample page". Similarly, the `<body>` element contains an `<h1>` element, a `<p>` element, and a comment.

This DOM tree can be manipulated from scripts in the page. Scripts (typically in JavaScript) are small programs that can be embedded using the `<script>` element or using event handler content attributes. For example, here is a form with a script that sets the value of the form's `<output>` element to say "Hello World"

```
<form name="main">
  Result: <output name="result"></output>
  <script>
    document.forms.main.elements.result.value = 'Hello World';
  </script>
</form>
```

Each element in the DOM tree is represented by an object, and these objects have APIs so that they can be manipulated. For instance, a link (e.g., the `<a>` element in the tree above) can have its "`href`" attribute changed in several ways:

```
var a = document.links[0]; // obtain the first link in the document
a.href = 'sample.html'; // change the destination URL of the link
a.protocol = 'https'; // change just the scheme part of the URL
a.setAttribute('href', 'http://example.com/'); // change the content attribute directly
```

Since DOM trees are used as the way to represent HTML documents when they are processed and presented by implementations (especially interactive implementations like Web browsers), this specification is mostly phrased in terms of DOM trees, instead of the markup described above.

HTML documents represent a media-independent description of interactive content. HTML documents might be rendered to a screen, or through a speech synthesizer, or on a braille display. To influence exactly how such rendering takes place, authors can use a styling language such as CSS.

In the following example, the page has been made yellow-on-blue using CSS.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Sample styled page</title>
    <style>
      body { background: navy; color: yellow; }
    </style>
  </head>
  <body>
    <h1>Sample styled page</h1>
    <p>This page is just a demo.</p>
  </body>
</html>
```

For more details on how to use HTML, authors are encouraged to consult tutorials and guides. Some of the examples included in this specification might also be of use, but the novice author is cautioned that this specification, by necessity, defines the language with a level of detail that might be difficult to understand at first.

§ 1.9.1. Writing secure applications with HTML

This section is non-normative.

When HTML is used to create interactive sites, care needs to be taken to avoid introducing vulnerabilities through which attackers can compromise the integrity of the site itself or of the site's users.

A comprehensive study of this matter is beyond the scope of this document, and authors are strongly

encouraged to study the matter in more detail. However, this section attempts to provide a quick introduction to some common pitfalls in HTML application development.

The security model of the Web is based on the concept of "origins", and correspondingly many of the potential attacks on the Web involve cross-origin actions. [ORIGIN]

Not validating user input
Cross-site scripting (XSS)
SQL injection

When accepting untrusted input, e.g., user-generated content such as text comments, values in URL parameters, messages from third-party sites, etc, it is imperative that the data be validated before use, and properly escaped when displayed. Failing to do this can allow a hostile user to perform a variety of attacks, ranging from the potentially benign, such as providing bogus user information like a negative age, to the serious, such as running scripts every time a user looks at a page that includes the information, potentially propagating the attack in the process, to the catastrophic, such as deleting all data in the server.

When writing filters to validate user input, it is imperative that filters always be safelist-based, allowing known-safe constructs and disallowing all other input. Blocklist-based filters that disallow known-bad inputs and allow everything else are not secure, as not everything that is bad is yet known (for example, because it might be invented in the future).

¶ EXAMPLE COUNTER(EXAMPLE) For example, suppose a page looked at its URL's query string to determine what to display, and the site then redirected the user to that page to display a message, as in:

```
<ul>
  <li><a href="message.cgi?say=Hello">Say Hello</a>
  <li><a href="message.cgi?say=Welcome">Say Welcome</a>
  <li><a href="message.cgi?say=Kittens">Say Kittens</a>
</ul>
```

If the message was just displayed to the user without escaping, a hostile attacker could then craft a URL that contained a script element:

```
http://example.com/message.cgi?say=%3Cscript%3Ealert%28%27Oh%20no%21%27%29%3C/script%3E
```

If the attacker then convinced a victim user to visit this page, a script of the attacker's choosing would run on the page. Such a script could do any number of hostile actions, limited only by what the site offers: if the site is an e-commerce shop, for instance, such a script could cause the user to unknowingly make arbitrarily many unwanted purchases.

This is called a cross-site scripting attack.

There are many constructs that can be used to try to trick a site into executing code. Here are some that authors are encouraged to consider when writing safelist filters:

- When allowing harmless-seeming elements like `<img>`, it is important to safelist any provided attributes as well. If one allowed all attributes then an attacker could, for instance, use the `onload` attribute to run arbitrary script.
- When allowing URLs to be provided (e.g., for links), the scheme of each URL also needs to be explicitly safelisted, as there are many schemes that can be abused. The most prominent example is "`javascript:`", but user agents can implement (and indeed, have historically implemented) others.
- Allowing a `<base>` element to be inserted means any `<script>` elements in the page with relative links

can be hijacked, and similarly that any form submissions can get redirected to a hostile site.

Cross-site request forgery (CSRF)

If a site allows a user to make form submissions with user-specific side-effects, for example posting messages on a forum under the user's name, making purchases, or applying for a passport, it is important to verify that the request was made by the user intentionally, rather than by another site tricking the user into making the request unknowingly.

This problem exists because HTML forms can be submitted to other origins.

Sites can prevent such attacks by populating forms with user-specific hidden tokens, or by checking `Origin` headers on all requests.

Clickjacking

A page that provides users with an interface to perform actions that the user might not wish to perform needs to be designed so as to avoid the possibility that users can be tricked into activating the interface.

One way that a user could be so tricked is if a hostile site places the victim site in a small `<iframe>` and then convinces the user to click, for instance by having the user play a reaction game. Once the user is playing the game, the hostile site can quickly position the `<iframe>` under the mouse cursor just as the user is about to click, thus tricking the user into clicking the victim site's interface.

To avoid this, sites that do not expect to be used in frames are encouraged to only enable their interface if they detect that they are not in a frame (e.g., by comparing the `window` object to the value of the `top` attribute).

§ 1.9.2. Common pitfalls to avoid when using the scripting APIs

This section is non-normative.

Scripts in HTML have "run-to-completion" semantics, meaning that the browser will generally run the script uninterrupted before doing anything else, such as firing further events or continuing to parse the document.

On the other hand, parsing of HTML files happens incrementally, meaning that the parser can pause at any point to let scripts run. This is generally a good thing, but it does mean that authors need to be careful to avoid hooking event handlers after the events could have possibly fired.

There are two techniques for doing this reliably: use event handler content attributes, or create the element and add the event handlers in the same script. The latter is safe because, as mentioned earlier, scripts are run to completion before further events can fire.

¶ EXAMPLE COUNTER(EXAMPLE) One way this could manifest itself is with `<img>` elements and the `load` event. The event could fire as soon as the element has been parsed, especially if the image has already been cached (which is common).

Here, the author uses the `onload` handler on an `<img>` element to catch the `load` event:

```
<img src="games.png" alt="Games" onload="gamesLogoHasLoaded(event)">
```

If the element is being added by script, then so long as the event handlers are added in the same script, the event will still not be missed:

```
<script>
var img = new Image();
img.src = 'games.png';
img.alt = 'Games';
img.onload = gamesLogoHasLoaded;
// img.addEventListener('load', gamesLogoHasLoaded, false); // would work also
</script>
```

However, if the author first created the `<img>` element and then in a separate script added the event listeners, there's a chance that the `load` event would be fired in between, leading it to be missed:

```
<!-- Do not use this style, it has a race condition! -->
<img id="games" src="games.png" alt="Games">
<!-- the 'load' event might fire here while the parser is taking a
break, in which case you will not see it! -->
<script>
var img = document.getElementById('games');
img.onload = gamesLogoHasLoaded; // might never fire!
</script>
```

§ 1.9.3. How to catch mistakes when writing HTML: validators and conformance checkers

This section is non-normative.

Authors are encouraged to make use of conformance checkers (also known as *validators*) to catch common mistakes. The W3C provides a number of online validation services, including the Nu Markup Validation Service.

§ 1.10. Conformance requirements for authors

This section is non-normative.

Unlike previous versions of the HTML specification, this specification defines in some detail the required processing for invalid documents as well as valid documents.

However, even though the processing of invalid content is in most cases well-defined, conformance requirements for documents are still important: in practice, interoperability (the situation in which all implementations process particular content in a reliable and identical or equivalent way) is not the only goal of document conformance requirements. This section details some of the more common reasons for still distinguishing between a conforming document and one with errors.

§ 1.10.1. Presentational markup

This section is non-normative.

The majority of presentational features from previous versions of HTML are no longer allowed. Presentational markup in general has been found to have a number of problems:

The use of presentational elements leads to poorer accessibility
While it is possible to use presentational markup in a way that provides users of assistive technologies (ATs) with an acceptable experience (e.g., using ARIA), doing so is significantly more difficult than doing so when using semantically-appropriate markup. Furthermore, even using such techniques doesn't help make pages accessible for non-AT, non-graphical users, such as users of text-mode browsers.

Using media-independent markup, on the other hand, provides an easy way for documents to be authored in such a way that they are "accessible" for more users (e.g., users of text browsers).

Higher cost of maintenance
It is significantly easier to maintain a site written in such a way that the markup is style-independent. For example, changing the color of a site that uses `<font color="">` throughout requires changes across the entire site, whereas a similar change to a site based on CSS can be done by changing a single file.

Larger document sizes
Presentational markup tends to be much more redundant, and thus results in larger document sizes.

For those reasons, presentational markup has been removed from HTML in this version. This change should not come as a surprise; HTML 4.0 deprecated presentational markup many years ago and provided a mode (HTML Transitional) to help authors move away from presentational markup; later, XHTML 1.1 went further and obsoleted those features altogether.

The only remaining presentational markup features in HTML are the `style` attribute and the `<style>` elemen t. Use of the `style` attribute is somewhat discouraged in production environments, but it can be useful for rapid prototyping (where its rules can be directly moved into a separate style sheet later) and for providing specific styles in unusual cases where a separate style sheet would be inconvenient. Similarly, the `<style>` el ement can be useful in syndication or for page-specific styles, but in general an external style sheet is likely to be more convenient when the styles apply to multiple pages.

It is also worth noting that some elements that were previously presentational have been redefined in this specification to be media-independent: `<b>`, `<i>`, `<hr>`, `<s>`, `<small>`, and `<u>`.

§ 1.10.2. Syntax errors

This section is non-normative.

The syntax of HTML is constrained to avoid a wide variety of problems.

Unintuitive error-handling behavior
Certain invalid syntax constructs, when parsed, result in DOM trees that are highly unintuitive.

¶ EXAMPLE COUNTER(EXAMPLE) For example, the following markup fragment results in a DOM with an `<hr>` element that is an *earlier* sibling of the corresponding `<table>` element:

```
<table><hr>...
```

Errors with optional error recovery

To allow user agents to be used in controlled environments without having to implement the more bizarre and convoluted error handling rules, user agents are permitted to fail whenever encountering a parse error.

Errors where the error-handling behavior is not compatible with streaming user agents

Some error-handling behavior, such as the behavior for the `<table><hr>...` example mentioned above, are incompatible with streaming user agents (user agents that process HTML files in one pass, without storing state). To avoid interoperability problems with such user agents, any syntax resulting in such behavior is considered invalid.

Errors that can result in infoset coercion

When a user agent based on XML is connected to an HTML parser, it is possible that certain invariants that XML enforces, such as element or attribute names never contain multiple colons, will be violated by an HTML file. Handling this can require that the parser coerce the HTML DOM into an XML-compatible infoset. Most syntax constructs that require such handling are considered invalid. (Comments containing two consecutive hyphens, or ending with a hyphen, are exceptions that are allowed in the HTML syntax.)

Errors that result in disproportionately poor performance

Certain syntax constructs can result in disproportionately poor performance. To discourage the use of such constructs, they are typically made non-conforming.

¶ EXAMPLE COUNTER(EXAMPLE) For example, the following markup results in poor performance, since all the unclosed `<i>` elements have to be reconstructed in each paragraph, resulting in progressively more elements in each paragraph:

```
<p><i>He dreamt.
<p><i>He dreamt that he ate breakfast.
<p><i>Then lunch.
<p><i>And finally dinner.
```

The resulting DOM for this fragment would be:

- `<p>`
  - `<i>`
    - `#text`: He dreamt.
- `<p>`
  - `<i>`
    - `<i>`
      - `#text`: He dreamt that he ate breakfast.
- `<p>`
  - `<i>`
    - `<i>`
      - `<i>`
        - `#text`: Then lunch.
- `<p>`
  - `<i>`
    - `<i>`
      - `<i>`
        - `<i>`
          - `#text`: And finally dinner.

Errors involving fragile syntax constructs

There are syntax constructs that, for historical reasons, are relatively fragile. To help reduce the number of users who accidentally run into such problems, they are made non-conforming.

¶ EXAMPLE COUNTER(EXAMPLE) For example, the parsing of certain named character references in attributes happens even with the closing semicolon being omitted. It is safe to include an ampersand followed by letters that do not form a named character reference, but if the letters are changed to a string that *does* form a named character reference, they will be interpreted as that character instead.

In this fragment, the attribute's value is "`?bill&ted`":

```
<a href="?bill&ted">Bill and Ted</a>
```

In the following fragment, however, the attribute's value is actually "`?art©`", *not* the intended "`?art&copy`", because even without the final semicolon, "`&copy`" is handled the same as "`©`" and thus gets interpreted as "`©`":

```
<a href="?art&copy">Art and Copy</a>
```

To avoid this problem, all named character references are required to end with a semicolon, and uses of named character references without a semicolon are flagged as errors.

Thus, the correct way to express the above cases is as follows:

```
<a href="?bill&ted">Bill and Ted</a> <!-- &ted is ok, since it's not a named character refe
```

```
<a href="?art&amp;copy">Art and Copy</a> <!-- the & has to be escaped, since &copy is a nam
```

Errors involving known interoperability problems in legacy user agents

Certain syntax constructs are known to cause especially subtle or serious problems in legacy user agents, and are therefore marked as non-conforming to help authors avoid them.

¶ EXAMPLE COUNTER(EXAMPLE) For example, this is why the U+0060 GRAVE ACCENT character (`) is not allowed in unquoted attributes. In certain legacy user agents, it is sometimes treated as a quote character.

¶ EXAMPLE COUNTER(EXAMPLE) Another example of this is the DOCTYPE, which is required to trigger no-quirks mode, because the behavior of legacy user agents in quirks mode is often largely undocumented.

Errors that risk exposing authors to security attacks

Certain restrictions exist purely to avoid known security problems.

¶ EXAMPLE COUNTER(EXAMPLE) For example, the restriction on using UTF-7 exists purely to avoid authors falling prey to a known cross-site-scripting attack using UTF-7. [RFC2152]

Cases where the author's intent is unclear

Markup where the author's intent is very unclear is often made non-conforming. Correcting these errors early makes later maintenance easier.

¶ EXAMPLE COUNTER(EXAMPLE) For example, it is unclear whether the author intended the following to be an `<h1>` heading or an `<h2>` heading:

```
<h2>Contact details</h1>
```

Cases that are likely to be typos

When a user makes a simple typo, it is helpful if the error can be caught early, as this can save the author a lot of debugging time. This specification therefore usually considers it an error to use element names, attribute names, and so forth, that do not match the names defined in this specification.

¶ EXAMPLE COUNTER(EXAMPLE) For example, if the author typed `<capton>` instead of `<caption>`, this would be flagged as an error and the author could correct the typo immediately.

Errors that could interfere with new syntax in the future

In order to allow the language syntax to be extended in the future, certain otherwise harmless features are disallowed.

¶ EXAMPLE COUNTER(EXAMPLE) For example, attributes in end tags are ignored currently, but they are invalid, in case a future change to the language makes use of that syntax feature without conflicting with already-deployed (and valid!) content.

Some authors find it helpful to be in the practice of always quoting all attributes and always including all optional tags, preferring the consistency derived from such custom over the minor benefits of terseness afforded by making use of the flexibility of the HTML syntax. To aid such authors, conformance checkers can provide modes of operation wherein such conventions are enforced.

§ 1.10.3. Restrictions on content models and on attribute values

This section is non-normative.

Beyond the syntax of the language, this specification also places restrictions on how elements and attributes can be specified. These restrictions are present for similar reasons:

Errors involving content with dubious semantics

To avoid misuse of elements with defined meanings, content models are defined that restrict how elements can be nested when such nestings would be of dubious value.

¶ EXAMPLE COUNTER(EXAMPLE) For example, this specification disallows nesting a `<section>` elem ent inside a `<kbd>` element, since it is highly unlikely for an author to indicate that an entire section should be keyed in.

Errors that involve a conflict in expressed semantics

Similarly, to draw the author's attention to mistakes in the use of elements, clear contradictions in the semantics expressed are also considered conformance errors.

¶ EXAMPLE COUNTER(EXAMPLE) In the fragments below, for example, the semantics are nonsensical: a separator cannot simultaneously be a cell, nor can a radio button be a progress bar.

```
<hr role="cell">
```

```
<input type=radio role=progressbar>
```

¶ EXAMPLE COUNTER(EXAMPLE) Another example is the restrictions on the content models of the `<ul>` element, which only allows `<li>` element children. Lists by definition consist just of zero or more list items, so if a `<ul>` element contains something other than an `<li>` element, it's not clear what was meant.

Cases where the default styles are likely to lead to confusion

Certain elements have default styles or behaviors that make certain combinations likely to lead to confusion. Where these have equivalent alternatives without this problem, the confusing combinations are disallowed.

¶ EXAMPLE COUNTER(EXAMPLE) For example, `<div>` elements are rendered as block boxes, and `<span>` elements as inline boxes. Putting a block box in an inline box is unnecessarily confusing; since either nesting just `<div>` elements, or nesting just `<span>` elements, or nesting `<span>` elements inside `<div>` elements all serve the same purpose as nesting a `<div>` element in a `<span>` element, but only the latter involves a block box in an inline box, the latter combination is disallowed.

¶ EXAMPLE COUNTER(EXAMPLE) Another example would be the way interactive content cannot be nested. For example, a `<button>` element cannot contain a `<textarea>` element. This is because the default behavior of such nesting interactive elements would be highly confusing to users. Instead of nesting these elements, they can be placed side by side.

Errors that indicate a likely misunderstanding of the specification

Sometimes, something is disallowed because allowing it would likely cause author confusion.

¶ EXAMPLE COUNTER(EXAMPLE) For example, setting the `disabled` attribute to the value "`false`" is disallowed, because despite the appearance of meaning that the element is enabled, it in fact means that the element is *disabled* (what matters for implementations is the presence of the attribute, not its value).

Errors involving limits that have been imposed merely to simplify the language

Some conformance errors simplify the language that authors need to learn.

¶ EXAMPLE COUNTER(EXAMPLE) For example, the `<area>` element's `shape` attribute, despite accepting both "`circ`" and "`circle`" values in practice as synonyms, disallows the use of the "`circ`" value, so as to simplify tutorials and other learning aids. There would be no benefit to allowing both, but it would cause extra confusion when teaching the language.

Errors that involve peculiarities of the parser

Certain elements are parsed in somewhat eccentric ways (typically for historical reasons), and their content model restrictions are intended to avoid exposing the author to these issues.

¶ EXAMPLE COUNTER(EXAMPLE) For example, a `<form>` element isn't allowed inside phrasing content, because when parsed as HTML, a `<form>` element's start tag will imply a `<p>` element's end tag. Thus, the following markup results in two paragraphs, not one:

```
<p>Welcome. <form><label>Name:</label> <input></form>
```

It is parsed exactly like the following:

```
<p>Welcome. </p><form><label>Name:</label> <input></form>
```

Errors that would likely result in scripts failing in hard-to-debug ways

Some errors are intended to help prevent script problems that would be hard to debug.

¶ EXAMPLE COUNTER(EXAMPLE) This is why, for instance, it is non-conforming to have two `id` attributes with the same value. Duplicate IDs lead to the wrong element being selected, with sometimes disastrous effects whose cause is hard to determine.

Errors that waste authoring time

Some constructs are disallowed because historically they have been the cause of a lot of wasted authoring time, and by encouraging authors to avoid making them, authors can save time in future efforts.

¶ EXAMPLE COUNTER(EXAMPLE) For example, a `<script>` element's `src` attribute causes the element's contents to be ignored. However, this isn't obvious, especially if the element's contents appear to be executable script — which can lead to authors spending a lot of time trying to debug the inline script without realizing that it is not executing. To reduce this problem, this specification makes it non-conforming to have executable script in a `<script>` element when the `src` attribute is present. This means that authors who are validating their documents are less likely to waste time with this kind of mistake.

Errors that involve areas that affect authors migrating to and from XHTML

Some authors like to write files that can be interpreted as both XML and HTML with similar results. Though this practice is discouraged in general due to the myriad of subtle complications involved (especially when involving scripting, styling, or any kind of automated serialization), this specification has a few restrictions intended to at least somewhat mitigate the difficulties. This makes it easier for authors to use this as a transitionary step when migrating between HTML and XHTML.

¶ EXAMPLE COUNTER(EXAMPLE) For example, there are somewhat complicated rules surrounding the `lang` and `xml:lang` attributes intended to keep the two synchronized.

¶ EXAMPLE COUNTER(EXAMPLE) Another example would be the restrictions on the values of `xmlns` attributes in the HTML serialization, which are intended to ensure that elements in conforming documents end up in the same namespaces whether processed as HTML or XML.

Errors that involve areas reserved for future expansion

As with the restrictions on the syntax intended to allow for new syntax in future revisions of the language, some restrictions on the content models of elements and values of attributes are intended to allow for future expansion of the HTML vocabulary.

¶ EXAMPLE COUNTER(EXAMPLE) For example, limiting the values of the `target` attribute that start with an U+005F LOW LINE character (_) to only specific predefined values allows new predefined values to be introduced at a future time without conflicting with author-defined values.

Errors that indicate a mis-use of other specifications

Certain restrictions are intended to support the restrictions made by other specifications.

¶ EXAMPLE COUNTER(EXAMPLE) For example, requiring that attributes that take media query lists use only *valid* media query lists reinforces the importance of following the conformance rules of that specification.

§ 1.11. Suggested reading

This section is non-normative.

The following documents might be of interest to readers of this specification.

Character Model for the World Wide Web 1.0: Fundamentals [CHARMOD]

> This Architectural Specification provides authors of specifications, software developers, and content developers with a common reference for interoperable text manipulation on the World Wide Web, building on the Universal Character Set, defined jointly by the Unicode specification and ISO/IEC 10646. Topics addressed include use of the terms "character", "encoding" and "string", a reference processing model, choice and identification of character encodings, character escaping, and string indexing.

Unicode Security Considerations [UNICODE-SECURITY]

> Because Unicode contains such a large number of characters and incorporates the varied writing systems of the world, incorrect usage can expose programs or systems to possible security attacks. This is especially important as more and more products are internationalized. This document describes some of the security considerations that programmers, system analysts, standards developers, and users should take into account, and provides specific recommendations to reduce the risk of problems.

Web Content Accessibility Guidelines (WCAG) 2.0 [WCAG20]

> Web Content Accessibility Guidelines (WCAG) 2.0 covers a wide range of recommendations for making Web content more accessible. Following these guidelines will make content accessible to a wider range of people with disabilities, including blindness and low vision, deafness and hearing loss, learning disabilities, cognitive limitations, limited movement, speech disabilities, photosensitivity and combinations of these. Following these guidelines will also often make your Web content more usable to users in general.

Authoring Tool Accessibility Guidelines (ATAG) 2.0 [ATAG20]

> This specification provides guidelines for designing Web content authoring tools that are more accessible for people with disabilities. An authoring tool that conforms to these guidelines will promote accessibility by providing an accessible user interface to authors with disabilities as well as by enabling, supporting, and promoting the production of accessible Web content by all authors.

User Agent Accessibility Guidelines (UAAG) 2.0 [UAAG20]

> This document provides guidelines for designing user agents that lower barriers to Web accessibility for people with disabilities. User agents include browsers and other types of software that retrieve and render Web content. A user agent that conforms to these guidelines will promote accessibility through its own user interface and through other internal facilities, including its ability to communicate with other technologies (especially assistive technologies). Furthermore, all users, not just users with disabilities, should find conforming user agents to be more usable.

HTML Accessibility APIs Mappings 1.0 [html-aam-1.0]

> Defines how user agents map HTML 5.1 elements and attributes to platform accessibility APIs. Documenting these mappings promotes interoperable exposure of roles, states, properties, and events implemented by accessibility APIs and helps to ensure that this information appears in a manner consistent with author intent.